SOFTWARE REQUIREMENTS SPECIFICATION

Arya College ClubSphere: Centralized Club and Event Management Platform

Prepared in IEEE 830-style SRS structure

Submitted in partial fulfillment of the requirements for the

Bachelor of Technology in Computer Science & Engineering

Submitted to:

Rajasthan Technical University, Kota (Rajasthan)

Project Guide

Er. Ram Babu Buri

Department of Computer Science & Engineering

Arya College of Engineering & I.T.

SP-42, RIICO Industrial Area, Kukas, Jaipur

PBL Team ID: PBL2627-CSE-A-024

Version 1.0 | Academic Session 2026–27

Prepared by

Avantika Mishra (24EARCS032) · Avanshi Kothari (24EARCS031)
Akansha Sharma (24EARCS012) · Aditi Maheshwari (24EARCS007)

Table of Contents

- Document Control
- 1. Introduction
- 2. Overall Description
- 3. External Interface Requirements
- 4. System Features and Functional Requirements
- 5. Non-Functional Requirements
- 6. Logical Database Requirements
- 7. Verification and Acceptance Criteria
- Appendix A. Use Case Summary
- Appendix B. Current Scope Versus Planned Scope
- Appendix C. Open Issues Before Final Baseline

Document Control

Field	Value
Project	Arya College ClubSphere: Centralized Club and Event Management Platform
Document	Software Requirements Specification (SRS)
Version	1.0
Prepared for	Project-Based Learning (PBL)
PBL Team ID	PBL2627-CSE-A-024
Institution	Arya College of Engineering & I.T., Jaipur
University	Rajasthan Technical University, Kota
Project Guide	Er. Ram Babu Buri
Session	2026–27
Status	Requirements baseline for the current college-focused release

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the requirements for Arya College ClubSphere, a web-based club and event management portal developed for Arya College of Engineering & I.T., Jaipur. It describes the system scope, users, interfaces, functional behaviour, quality requirements, constraints, and acceptance criteria. It is intended for the project team, project guide, evaluators, testers, and future maintainers.

This document follows the traditional IEEE 830-style SRS organization. Requirements distinguish the current implementation from planned capabilities; a feature must not be considered implemented merely because it is specified here.

1.2 Scope

ClubSphere aims to give administrators, club heads, club members and students a common web entry point for the official club directory, event publishing, QR-based registration, club tasks, announcements and in-app notifications. The current release focuses on session login with four roles (ADMIN, CLUB_HEAD, CLUB_MEMBER, STUDENT), BCrypt password hashing, MySQL persistence, a Tomcat WAR, and an optional Docker demo.

Out of scope for this release: public self-service signup (administrators create accounts), native mobile apps, payment gateways, email/SMS gateways, a JSON REST API for a separate SPA, and LMS features such as courses or assignments. ClubSphere is not a general campus academic portal.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
BCrypt	Password-hashing algorithm used to store password verifiers rather than plaintext passwords.
DAO	Data Access Object; JDBC class that executes parameterized SQL.
JDBC	Java Database Connectivity.
JSP	Jakarta Server Pages used to render HTML.
PBL	Project-Based Learning.
QR	Quick Response code encoding an event registration URL.
RBAC	Role-Based Access Control based on the authenticated account’s role.
SRS	Software Requirements Specification.

Term	Definition
WAR	Web Application Archive deployed on Apache Tomcat.
ADMIN / CLUB_HEAD / CLUB_MEMBER / STUDENT	The four application roles.

1.4 References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
- Arya College ClubSphere Project Report, Academic Session 2026–27, PBL2627-CSE-A-024.
- Jakarta Servlet 6.0 Specification / Apache Tomcat 10.1 Documentation.
- MySQL 8.0 Reference Manual: https://dev.mysql.com/doc/
- Apache Maven Documentation: https://maven.apache.org/guides/
- ZXing barcode library: https://github.com/zxing/zxing
- Git Documentation: https://git-scm.com/doc
- Project repository: https://github.com/Avantikamishra862/Arya_Clubsphere
- UML diagram set: docs/UML/Arya_College_ClubSphere_UML_Diagrams.pdf

1.5 Overview

Section 2 describes product context, functions, users, environment, constraints and assumptions. Section 3 lists external interfaces. Section 4 specifies functional requirements. Section 5 specifies non-functional requirements. Section 6 records logical data requirements. Section 7 defines verification and acceptance. Appendices summarise use cases, current versus planned scope, and open issues.

2. Overall Description

2.1 Product Perspective

ClubSphere is a browser-based, server-rendered Java web application. Apache Tomcat 10.1 hosts JSP views and Jakarta Servlets. AuthenticationFilter enforces a public whitelist and role prefixes. Servlets call services; DAOs access MySQL 8 through JDBC and HikariCP. This is not a React SPA and does not use JWT.

2.2 Product Functions

- Authenticate registered users with username or email and BCrypt-verified passwords.
- Maintain a server-side HTTP session (currentUser) after login; invalidate it on logout.
- Distinguish ADMIN, CLUB_HEAD, CLUB_MEMBER and STUDENT and apply RBAC to URL prefixes.
- Allow administrators to create users, change roles and toggle account status.
- Present a public club directory and event catalogue.
- Allow club heads to manage members, events, QR codes, tasks and announcements for the assigned club.
- Allow students to register for events by eventId or QR token (individual or group).
- Provide role dashboards, in-app notifications and simple reports.
- Persist application data in MySQL (clubsphere_db).
- Support local WAR deployment, Docker Compose demo, and hosted Tomcat (PORT binding).

2.3 User Classes and Characteristics

User class	Description	Access principle
ADMIN	Faculty or student coordinator managing users, clubs and system-wide reports.	Only /admin/* plus public pages.
CLUB_HEAD	Lead of one assigned club.	/club-head/* for that club; may also use member/student areas.
CLUB_MEMBER	Active volunteer of a club.	/club-member/* for tasks and announcements.
STUDENT	Enrolled student browsing clubs and events.	/student/* plus public catalogue and registration.
Evaluator	Faculty or teammate demonstrating the system.	Uses seeded demo accounts documented in the project.

2.4 Operating Environment

- Client: modern web browser on a desktop or mobile device.
- Application: Java 17, Jakarta Servlets, JSP, Bootstrap 5, Tomcat 10.1.
- Database: MySQL 8, database name clubsphere_db.
- Security: BCrypt, HttpOnly session cookie, AuthenticationFilter RBAC.
- QR: ZXing PNG generation; phone camera optional for scanning.
- Build: Apache Maven 3.9+; optional Docker Engine for Compose.
- Development tools: Git/GitHub and a Java-capable editor.

2.5 Design and Implementation Constraints

- The project is developed within an academic PBL timeline (PBL2627-CSE-A-024).
- The deployment target is Arya College of Engineering & I.T., Jaipur.
- Tomcat 10.1 requires the Jakarta namespace (not javax.servlet).
- Database credentials shall be supplied through environment-specific configuration and shall not be committed as production secrets.
- Authorization must not depend only on hiding UI links.

- QR payload URLs depend on APP_BASE_URL matching the public origin and context path.
- Planned features must be labelled planned until implementation and tests are confirmed.
- Physical schema names must be verified against database/schema.sql.

2.6 Assumptions and Dependencies

- MySQL Server is installed, running, and reachable.
- Schema and seed scripts are applied before first demonstration.
- Java, Maven and (if used) Docker versions match the README.
- APP_BASE_URL is set correctly for local WAR versus Docker ROOT.war.
- The repository README is authoritative for exact commands and environment variable names.

3. External Interface Requirements

3.1 User Interfaces

The system shall provide a public landing page, login screen, club explore/view pages, event list/view pages, and role-specific dashboards after authentication. Forms shall label fields clearly, indicate validation errors in text, and display useful success or failure feedback. Bootstrap 5 shall support common laptop and mobile viewports. The login page may include evaluator autofill for seeded demo accounts.

3.2 Hardware Interfaces

No dedicated hardware is required. Optional QR scanning uses a phone camera or a downloaded PNG. Users access the system through a device capable of running a supported web browser.

3.3 Software Interfaces

Interface	Requirement
Browser to Tomcat	HTTP form posts and page loads; JSP HTML responses.
Login	POST /login with username/email and password; GET /logout invalidates the session.
QR image	GET /qr/{token} returns image/png generated by ZXing.
Registration	GET/POST /event/register with eventId or token; login required.
Application to MySQL	JDBC via HikariCP using DB_URL / DB_USERNAME / DB_PASSWORD (or application.properties placeholders).
Build	Maven WAR aryaClgClubSphere.war; Docker image deploys as ROOT.war and binds $PORT.

3.4 Communications Interfaces

The browser and Tomcat communicate over HTTP during local development and should use HTTPS on the hosted demo. Session cookies shall be HttpOnly. Secure shall be enabled when serving only over HTTPS. Credentials, password hashes and internal stack traces shall not appear in ordinary user-facing error pages.

4. System Features and Functional Requirements

The following requirements are uniquely identified for traceability. “Shall” indicates required behaviour for the specified release. Requirements that depend on a future capability are marked future.

4.1 Login and Session Authentication

FR-001 — The system shall authenticate a user by username or email plus password.

FR-002 — The system shall store passwords as BCrypt hashes and shall not persist plaintext passwords.

FR-003 — On successful login the system shall invalidate any previous HTTP session and create a new authenticated session.

FR-004 — After login the system shall redirect ADMIN, CLUB_HEAD, CLUB_MEMBER and STUDENT users to their respective dashboards.

FR-005 — Unauthenticated requests to protected URLs shall redirect to login, preserving a safe internal return path.

FR-006 — The login redirect parameter shall reject external URLs and javascript: targets.

FR-007 — Logout shall invalidate the session and return to the login page.

FR-008 — Invalid credentials shall not create an authenticated session.

4.2 Role-Based Access and User Administration

FR-009 — /admin/* shall be accessible only to the ADMIN role.

FR-010 — /club-head/* shall be accessible to CLUB_HEAD and ADMIN.

FR-011 — /club-member/* shall be accessible to CLUB_MEMBER, CLUB_HEAD and ADMIN.

FR-012 — Administrators shall create users, change roles, and toggle ACTIVE/INACTIVE/SUSPENDED status.

FR-013 — An administrator shall not deactivate their own currently logged-in account.

FR-014 — Hiding a menu link shall not be treated as authorization; the filter and servlets shall enforce access.

4.3 Club Directory and Membership

FR-015 — Visitors shall browse and search clubs by keyword and category without logging in.

FR-016 — Visitors shall open a club detail page including description, advisor, members and related events.

FR-017 — Administrators shall create, update and activate/deactivate clubs.

FR-018 — Each club may have at most one head user (head_user_id).

FR-019 — Club heads shall add and remove members of their assigned club only.

FR-020 — The system shall prevent duplicate memberships for the same user and club.

4.4 Events and QR Registration

FR-021 — Club heads shall create and edit events for their club, including schedule, venue, capacity, type and participation mode.

FR-022 — Each event shall have a unique qr_token.

FR-023 — GET /qr/{token} shall return a PNG QR code encoding the public registration URL built from APP_BASE_URL.

FR-024 — Students shall register using eventId or the QR token.

FR-025 — A user shall not register twice for the same event.

FR-026 — Group registrations shall store participant rows linked to the registration.

FR-027 — Club heads shall be able to cancel an event they own.

FR-028 — Club heads and administrators shall list registrations.

FR-029 — Authenticated students shall view and cancel their own registrations.

FR-030 — Unknown QR tokens shall not reveal other events; the system shall return not-found behaviour.

4.5 Dashboards, Tasks and Notifications

FR-031 — Each role shall have a dashboard summarising relevant counts and lists.

FR-032 — Club heads shall assign tasks with due date and priority.

FR-033 — Assignees shall update task status (PENDING, IN_PROGRESS, COMPLETED).

FR-034 — Club heads shall publish announcements with priority and audience (ALL or MEMBERS_ONLY).

FR-035 — Authenticated users shall see an in-app notification list and mark items read.

FR-036 — Administrators and club heads shall open a reports view backed by aggregated queries.

FR-037 — Successful login shall write an activity-log row.

4.6 Persistence and Configuration

FR-038 — The system shall persist users, clubs, events, registrations, tasks, announcements, notifications and activity logs in MySQL.

FR-039 — Database credentials shall be overridable by environment variables and shall not be committed as production secrets.

FR-040 — The application shall build with mvn clean package into a WAR suitable for Tomcat 10.1.

4.7 Future Features

FR-041 (future) — Public self-service student registration with college-email validation.

FR-042 (future) — Email or SMS reminders for deadlines and event changes.

FR-043 (future) — Attendance marking by a check-in QR distinct from the registration QR.

FR-044 (future) — Export of registration and report data (for example CSV).

5. Non-Functional Requirements

5.1 Security

NFR-SEC-001 — Passwords shall be stored as one-way BCrypt hashes and shall not be stored or logged in plaintext.

NFR-SEC-002 — Database credentials shall be provided through environment-specific configuration.

NFR-SEC-003 — Protected URLs shall validate the session and enforce server-side role checks.

NFR-SEC-004 — Error pages shall not expose stack traces, SQL statements or password hashes to end users.

NFR-SEC-005 — The hosted application should use HTTPS to protect data in transit.

NFR-SEC-006 — DAOs shall use parameterized SQL (PreparedStatement).

NFR-SEC-007 — Session cookies shall be HttpOnly; Secure should be set for HTTPS-only deployments.

5.2 Usability

NFR-USE-001 — Forms shall provide clear labels and understandable validation feedback.

NFR-USE-002 — Primary screens shall remain usable at common desktop and mobile viewport sizes.

NFR-USE-003 — Errors shall be communicated in text and shall not rely on colour alone.

5.3 Reliability

NFR-REL-001 — The application shall handle invalid input, authentication failures and database connectivity problems in a controlled way.

NFR-REL-002 — The project shall document repeatable startup steps for MySQL, Maven/WAR and Docker Compose.

NFR-REL-003 — Persisted data shall remain available after application restart if MySQL remains available.

5.4 Performance

NFR-PER-001 — Connection pooling (HikariCP) shall be used for JDBC access.

NFR-PER-002 — No numeric performance target is asserted because no measured baseline is claimed in the project report.

5.5 Maintainability, Portability and Configuration

NFR-MNT-001 — The application shall separate servlet, service and DAO responsibilities.

NFR-PORT-001 — Setup documentation shall identify required JDK 17, Maven, Tomcat 10.1 and MySQL 8.

NFR-CONF-001 — Environment-specific configuration shall be documented in .env.example without publishing real passwords.

NFR-CONF-002 — Docker shall bind Tomcat to $PORT (default 8080) and deploy the WAR as ROOT.war.

NFR-TEST-001 — Unit tests for password hashing and QR PNG generation shall run with mvn test without a live database.

6. Logical Database Requirements

The following conceptual model is aligned with database/schema.sql. Physical names, types, constraints and relationships in that script are normative for this release.

Entity	Conceptual attributes	Purpose / constraints
users	id, username, email, password_hash, full_name, role, student_id, status	Identity. Unique username and email. Role enum ADMIN / CLUB_HEAD / CLUB_MEMBER / STUDENT.
clubs	id, club_code, name, category, head_user_id, status	Official society. Unique club_code. Optional single head.
club_members	id, club_id, user_id, member_role, status	Roster. Unique (club_id, user_id).
events	id, title, club_id, dates/times, venue, capacity, qr_token, status	Campus event. Unique qr_token.
registrations	id, registration_number, event_id, user_id, type, status	Signup. Unique (event_id, user_id).
event_participants	id, registration_id, participant_name, is_leader	Group registration details.
tasks	id, club_id, event_id, assigned_to, due_date, status	Club work items.
announcements	id, club_id, title, content, priority, target_role	Club notices.
notifications	id, user_id, type, is_read	In-app inbox.
activity_logs	id, user_id, action, ip_address	Audit of actions such as login.

6.1 Data Integrity Requirements

- Primary identifiers shall uniquely identify records.
- Required fields shall be validated before persistence.
- Passwords shall be stored only as encoded hashes.
- Foreign keys shall maintain club, event and user relationships (InnoDB).
- Character set shall be utf8mb4.
- Deletion policy: operational practice prefers deactivating clubs/users; schema cascades exist and must be used with care.

7. Verification and Acceptance Criteria

The tests below are planned verification activities. Actual outcomes must be recorded after execution; no test is marked as passed merely because it appears in this SRS.

Test ID	Scenario	Expected result
TC-01	Login admin / seeded password	Admin dashboard is shown.
TC-02	Login with incorrect password	Error on login page; no session.
TC-03	Logout	Session invalidated; login page shown.
TC-04	Student GET /admin/dashboard	Access denied or login redirect.
TC-05	Admin creates a user with duplicate email	Creation rejected with a clear message.
TC-06	Admin attempts to deactivate own account	Operation blocked.
TC-07	Browse /clubs without login	Directory is visible.
TC-08	Club head adds a duplicate member	Rejected by uniqueness rule.
TC-09	Create event and GET /qr/{token}	PNG image returned.
TC-10	Register twice for the same event	Duplicate rejected; existing confirmation shown.

Test ID	Scenario	Expected result
TC-11	Register with unknown QR token	Not-found behaviour; no other event leaked.
TC-12	mvn test	PasswordUtil and QRCodeUtil tests run without MySQL.
TC-13	Stop MySQL and start the application	Connectivity failure is diagnosable without leaking the password.

7.1 Release Acceptance

- The WAR builds with documented Maven commands.
- MySQL schema and seed scripts run successfully.
- Login success and failure paths are tested for all four demo roles.
- RBAC is tested by attempting the wrong role URL.
- Club browse, event registration and QR PNG are demonstrated.
- Known limitations and unimplemented future modules are clearly documented.

Appendix A. Use Case Summary

Use case	Primary actor	Precondition	Outcome
Log in	Any registered user	Account exists and is ACTIVE.	Valid credentials create a session and open the role dashboard.
Manage users	Admin	Authenticated as ADMIN.	Users can be created; roles and status updated.
Browse clubs / events	Visitor or student	Public pages available.	Catalogue is displayed; filters work.
Manage club members	Club Head	User is head of a club.	Members added or removed for that club only.
Create event / show QR	Club Head	Managed club exists.	Event stored with qr_token; PNG available.
Register for event	Student	Logged in; event open.	Registration created once; group rows if applicable.
Assign / update task	Club Head / Member	Authenticated club user.	Task created or status updated; assignee notified.
Read notifications	Authenticated user	Session exists.	Inbox listed; items marked read.

Appendix B. Current Scope Versus Planned Scope

Area	Status	SRS treatment
Session login, BCrypt, RBAC filter	Implemented in the WAR; must be reverified in demonstration.	Core functional requirements FR-001–FR-014.
Club directory and membership	Implemented with 26 seeded clubs.	FR-015–FR-020.
Events, registrations, QR PNG	Implemented; APP_BASE_URL must match deploy context.	FR-021–FR-030.
Dashboards, tasks, notifications	Implemented as in-app features.	FR-031–FR-037.
Public self-registration	Not in this release (admins create users).	Future FR-041.
Email / SMS / attendance scan / CSV export	Not implemented.	Future FR-042–FR-044.
React / JWT / REST SPA	Out of product scope.	Not specified as ClubSphere requirements.

Appendix C. Open Issues Before Final Baseline

- Confirm APP_BASE_URL for the submission demonstration (local WAR versus Docker versus Render).
- Record actual TC-01–TC-13 outcomes on the running application and attach genuine screenshots if required by the department.
- Confirm that no production database password remains in source control.
- Agree whether public self-registration is required before the viva; if not, keep FR-041 as future.
- Set measurable performance targets only if the department requires them; none are claimed here.
- Keep UML, team report and this SRS consistent on role names and table names.